Learning Stage-wise Constraints from Expert Demonstrations

Baiyu Peng[1], Aude Billard[1]

***Abstract*—Learning task constraints from demonstrations provides an explicit and reusable representation for downstream planning. However, many existing constraint-learning methods assume that the same constraint set remains active throughout an entire trajectory, which is restrictive for multi-stage robotic tasks where different constraints govern different phases and the stage cutpoints are unobserved. This paper studies stage-wise constraint learning from feasible demonstrations. Given a task-dependent feature bank, the goal is to recover, for each stage and feature, whether the feature is inactive, equality-constrained, lower-bound constrained, or upper-bound constrained, together with the corresponding target or boundary value when active. We propose Stage-Wise Constraint Learning (SWCL) to jointly segments demonstrations and recovers stage-wise constraints. For each candidate segment, SWCL evaluates local feature evidence for equality, inequality, or inactive interpretations, subgoal-progress consistency, and cross-demonstration agreement with shared stage-wise prototypes. With fixed prototypes, each demonstration is segmented exactly by dynamic programming over ordered cutpoints; the prototypes are then updated by aggregating local modes and parameters across demonstrations. Experiments on planar obstacle avoidance, slide insertion, and spherical surface inspection show that SWCL recovers more accurate cutpoints and constraint parameters than HMM-, HSMM-, and clustering-based baselines, even though the baselines are given ground-truth active constraint pairs during parameter evaluation. Planning experiments further show that the learned constraints can be reused as feasibility conditions in modified and new task instances.**

***Index Terms*—Constraint learning, Learning from demonstration**

I. Introduction

Learning from demonstration (LfD) provides a practical way to specify robotic tasks when hand-engineering the desired behavior is difficult [1]. In many manipulation task, however, the reusable task specification is not a policy or a reward function, but a set of constraints that must be satisfied during execution. Such constraints are central to constrained motion planning and control: they specify alignment requirements, safety margins, speed limits, and other feasibility requirements that can be checked during downstream planning [2], [3]. Explicitly recovering these constraints from demonstrations is therefore useful because the learned constraints can be reused with new start states, goals, planners, or modified environments.

Many existing constraint-learning methods assume that the demonstrated task is governed by a constraint set that remains active and fixed throughout the whole trajectory. Representative examples include methods that infer constraints from locally optimal demonstrations using mixed-integer optimization, KKT conditions, or parametric and nonparametric constraint models [4]–[6], inverse constrained reinforcement learning or maximum-likelihood constraint inference methods [7]–[9], and recent positive-unlabeled formulation [10]. This global-constraint assumption is not well suited to multi-stage tasks, where different parts of a trajectory may be governed by different constraints. In slide insertion, for example, the constraints during free-space approach are different from those during contact, sliding, and final insertion. Applying a single global constraint model to such trajectories can mix incompatible regimes and obscure which constraints are actually active in each part of the task.

A separate line of work mainly studies task segmentation from demonstrations, often using HMMs, changepoint detection, or neural sequence-segmentation models [11]–[19]. These approaches can decompose a demonstrated trajectory into stages, skills, or motion primitives, but their outputs are typically local dynamics or controllers rather than explicit reusable constraints. More importantly, the segmentation model is usually not formulated around the explicit recovery of which constraints are active in each segment. As a result, the recovered stages are not necessarily aligned with changes in the underlying constraint structure. This leaves a gap between trajectory segmentation and constraint learning: for multi-stage tasks, one needs to infer both the latent stage cutpoints and the stage-wise constraint structure.

This paper addresses this gap by studying stage-wise constraint learning from feasible demonstrations. We assume that a task-dependent feature-function bank is given and that the true constraints are defined on a subset of these features. The objective is to recover which features are constrained at each stage, whether each active constraint is an equality, lower-bound inequality, or upper-bound inequality, and the corresponding target or boundary value. This is challenging because the stage cutpoints are unobserved. Additionally, demonstrations may have different start states, subgoals, final goals and trajectory lengths, while the underlying stage-wise constraint structure is shared.

We propose Stage-Wise Constraint Learning (SWCL), which formulates this problem as constraint-aware dynamic programming problem. The central idea is that segmentation and constraint recovery should inform each other: candidate segments should not only be temporally coherent, but should also support consistent feature-constraint interpretations across demonstrations. For every candidate segment, SWCL evaluates whether the feature traces support equality, inequality, or inactive modes; whether the motion progresses toward the segment endpoint; and whether the resulting local modes and parameters agree with shared stage-wise prototypes. With the current prototypes fixed, each demonstration is segmented exactly by

This work was supported by euROBIN Grant.

[1] The authors are with the LASA, School of Engineering, EPFL (Swiss Federal Institute of Technology in Lausanne), Lausanne 1015 Vaud, Switzerland. (baiyu.peng@epfl.ch ; aude.billard@epfl.ch). Corresponding author: Baiyu Peng.

dynamic programming over ordered cutpoints. The prototypes are then updated from the recovered segments by aggregating local constraint modes and parameters across demonstrations. This alternating procedure couples stage segmentation and constraint recovery while avoiding random initialization of latent state assignments or emission parameters.

We evaluate SWCL on three simulated multi-stage benchmark tasks: planar obstacle avoidance, slide insertion, and spherical surface inspection. We compare against three baselines adapted to stage-wise constraint recovery, and further demonstrate that the learned constraints can be reused for planning new trajectories in modified or new task instances.

II. Related Work

A. Learning Constraints from Demonstrations

Learning constraints from demonstrations has attracted increasing attention in robotics, inverse reinforcement learning, and safe planning. One line of work infers constraints from demonstrations by exploiting mixed-integer optimization, KKT conditions, or parametric and nonparametric constraint models. Chou et al. learn constraints from locally optimal demonstrations under cost uncertainty and later extend the idea to high-dimensional parametric constraints and Gaussian-process constraint models [4]–[6]. A second line formulates constraint inference through inverse constrained reinforcement learning or maximum-likelihood reasoning, where constraints explain why demonstrated behavior differs from unconstrained reward-maximizing behavior [7]–[9]. More recent work studies different supervision regimes, such as positive-unlabeled constraint learning from feasible demonstrations and generated unlabeled trajectories [10]. Equality-constraint learning has also been studied, for example by learning level-set representations of constraint manifolds for motion planning [20]. These methods recover explicit constraints, but they typically assume that the true constraint is global and time-invariant: the same constraint model applies throughout the whole trajectory. This assumption is restrictive for multi-stage tasks, where different constraints may become active in different stages. In contrast, our work aims to recover a stage-wise constraint mode matrix Γ, which specifies whether each feature is inactive, equality-constrained, lower-bound constrained, or upper-bound constrained in each stage, together with the corresponding constraint function parameters $\mathcal{P}$.

A smaller body of work explicitly connects task phases with local or stage-dependent constraints. Ureche et al. extract continuous task constraints from demonstrations and use them for task parameterization, including reference frames, relevant task variables, and stiffness modulation [21]. However, their method is mainly designed for equality-type constraints. Park et al. model demonstrations as a sequence of subtasks, each associated with a subgoal and locally active constraints, within a Bayesian nonparametric IRL framework [22]. However, this method assumes that the features in each stage are either constrained together or unconstrained together. Loula et al. learn stage-wise constraints by first decomposing demonstrations with a standard top-down segmentation procedure and then extracting constraints within each recovered segment [23]. This follows a sequential segmentation-then-constraint-extraction paradigm, where the segmentation step itself does not directly use constraint information. In contrast, SWCL learns segmentation and stage-wise constraints jointly. Other related works learn structured constraint representations for multi-step manipulation, such as keyframe-wise geometric constraint selection, dynamic constraint frames for hybrid force/position control, and contact-mode-specific kinematic constraints in contact-rich tasks [24]–[26]. These works rely on task-specific structure, such as keyframes, contact modes, or controller-level representations, and are therefore less general.

B. Task Segmentation and Sequential Imitation Learning

Task segmentation from demonstrations has a long history in robotics and time-series modeling. HMMs and HSMMs provide standard latent-state models for sequential data [11], [12]. In robotics, such models have been used to segment demonstrations into reusable skills, primitives, subgoals, or dynamical systems. Niekum et al. use Bayesian nonparametric autoregressive HMMs to recover finite-state task representations from unstructured demonstrations [15]. Medina and Billard combine HMMs with stable linear-parameter-varying systems to learn stable task sequences from demonstrations [16]. Task-parameterized HSMMs and related sequential imitation models have also been used to extract invariant segments and reproduce multi-step behaviors in new task situations [17], [27]. More generally, offline changepoint detection studies how to partition a sequence by optimizing an additive segment cost over ordered partitions [13], [14].

Recent learning-based methods further study task decomposition from weakly supervised or unsegmented demonstrations. TACO aligns demonstrations with weak task sketches while learning the corresponding sub-policies [18]. CompILE learns reusable variable-length behavior segments through a differentiable sequence-segmentation module [19]. More recently, language-guided skill learning uses large language models to propose initial trajectory segmentations, which are then refined through a hierarchical variational skill-discovery framework [28].

SWCL shares the ordered segmentation structure of these methods, but differs in its learning target. Standard segmentation and sequential imitation methods typically learn stage labels, stage cutpoints, local observation models, subgoals or controllers, rather than explicit reusable constraints. In SWCL, segmentation is instead an auxiliary problem used to recover the shared stage-wise constraint structure. The primary output is the learned constraint mode of each stage-feature pair, which can be inactive, equality-constrained, lower-bound constrained, or upper-bound constrained, together with the corresponding active constraint parameters. Accordingly, the segmentation objective is constraint-aware: candidate segments are evaluated not only by temporal or motion coherence, but also by local feature-constraint evidence and cross-demonstration agreement over the recovered constraint modes and parameters.

III. Preliminaries and Problem Formulation

We consider multi-stage tasks whose valid executions are characterized by known stage-independent feasibility require-

ments and unknown stage-dependent constraints on task features. Let $\mathcal{X} \subseteq \mathbb{R}^{d_x}$ denote the state space, and let $\Phi = \{\phi_j : \mathcal{X} \to \mathbb{R}\}_{j=1}^{F}$ be a given task-dependent feature bank. For a state x_t, the full feature vector is $\phi(x_t) = [\phi_1(x_t), \dots, \phi_F(x_t)]^\top \in \mathbb{R}^F$. Before learning, each feature dimension is standardized over the demonstrated data.

A task consists of K ordered stages. For a trajectory $\tau = \{x_t\}_{t=1}^{T}$, its stage schedule is represented by cutpoints $B = \{b_k\}_{k=0}^{K}$ satisfying $1 = b_0 < b_1 < \cdots < b_K = T + 1$. The cutpoints induce the stage intervals $\mathcal{I}_k(B) = \{b_{k-1}, \dots, b_k - 1\}$ and the corresponding stage endpoints $g_k(B) = x_{b_k - 1}$. When learning from demonstrations, the cutpoints and intermediate stage endpoints are unobserved and may differ across trajectories.

The unknown stage-wise constraint model is represented by a mode matrix $\Gamma = [\gamma_{k,j}] \in \{\varnothing, \mathrm{eq}, \mathrm{lb}, \mathrm{ub}\}^{K \times F}$. Here, $\gamma_{k,j} = \varnothing$ indicates that feature j is unconstrained in stage k, while eq, lb, and ub denote an equality constraint, a lower-bound inequality, and an upper-bound inequality, respectively. The constraint family of each feature is not specified in advance; every stage-feature pair is evaluated under all four modes.

For every active stage-feature pair, $\eta_{k,j} \in \mathbb{R}$ denotes its scalar constraint parameter. It specifies the target feature value for an equality constraint and the boundary value for an inequality constraint. The collection of active parameters is denoted by $\mathcal{P} = \{\eta_{k,j} : \gamma_{k,j} \neq \varnothing\}$. The mode and parameter of each stage-feature pair induce the feasible set

$$\mathcal{C}_{k,j} = \begin{cases} \mathcal{X}, & \gamma_{k,j} = \varnothing, \\ \{x \in \mathcal{X} : \phi_j(x) = \eta_{k,j}\}, & \gamma_{k,j} = \mathrm{eq}, \\ \{x \in \mathcal{X} : \phi_j(x) \geq \eta_{k,j}\}, & \gamma_{k,j} = \mathrm{lb}, \\ \{x \in \mathcal{X} : \phi_j(x) \leq \eta_{k,j}\}, & \gamma_{k,j} = \mathrm{ub}. \end{cases} \tag{1}$$

Given the stage-wise constraint model $(\Gamma, \mathcal{P})$, a trajectory length T, a start state x_{start}, and execution-specific ordered subgoals $\{g_k\}_{k=1}^{K}$, a valid task execution is defined as a solution to the following feasibility problem:

$$\begin{aligned} \text{find} \quad & \tau = \{x_t\}_{t=1}^{T}, \quad B = \{b_k\}_{k=0}^{K} \\ \text{s.t.} \quad & 1 = b_0 < b_1 < \cdots < b_K = T + 1, \\ & x_1 = x_{\mathrm{start}}, \\ & x_{b_k - 1} = g_k, \qquad k = 1, \dots, K, \\ & \tau \in \mathcal{C}_{\mathrm{known}}, \\ & x_t \in \bigcap_{j=1}^{F} \mathcal{C}_{k,j}, \qquad t \in \mathcal{I}_k(B), \quad k = 1, \dots, K. \end{aligned} \tag{2}$$

Here, $\mathcal{C}_{\mathrm{known}}$ contains known trajectory-level feasibility requirements, such as system dynamics and known collision-avoidance constraints.

We are given a set of feasible demonstrations $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^{N}$, where $\tau^{(i)} = \{x_t^{(i)}\}_{t=1}^{T_i}$. The demonstrations may have different lengths, start states, subgoals, and latent stage schedules, but share the same number and ordering of stages and the same stage-wise constraint model $(\Gamma, \mathcal{P})$. Each demonstration is assumed to satisfy (2) under its true latent schedule.

Definition 1 (Stage-wise constraint learning from demonstrations)**.** *Given feasible demonstrations $\mathcal{D}$, the number of stages K, and the feature bank Φ, stage-wise constraint learning aims to recover:*

1) *the shared constraint mode matrix $\Gamma = [\gamma_{k,j}]$, which specifies the active constraint mode of every stage-feature pair;*
2) *the active scalar constraint parameters $\mathcal{P} = \{\eta_{k,j} : \gamma_{k,j} \neq \varnothing\}$.*

The demonstration-specific stage schedules and their induced intermediate subgoals are not given and are jointly estimated during learning as auxiliary latent variables.

IV. Method

SWCL represents each latent task stage through a shared set of feature-wise constraint modes and parameters. Rather than describing a stage with a generic feature distribution, as is common in trajectory segmentation, SWCL evaluates each feature under four constraint-specific interpretations: inactive, equality-constrained, lower-bound constrained, and upper-bound constrained. The stage schedules are demonstration-specific, whereas the constraint mode and active constraint parameter of corresponding stages are shared across demonstrations.

Let $B^{(i)}$ denote the latent schedule of demonstration i, and let $\mathcal{I}_k^{(i)} = \mathcal{I}_k(B^{(i)})$ be its interval assigned to stage k. We use $\phi_j(x_t^{(i)})$ directly for the standardized observation of feature j at time t. In addition to the shared model $(\Gamma, \mathcal{P})$, let Ψ collect local distribution parameters, such as the feature locations and scales estimated separately for each demonstration-stage interval, and let $\boldsymbol{\kappa} = \{\kappa_k\}_{k=1}^{K}$ denote the parameters of an auxiliary motion-progress model. At a conceptual level, SWCL seeks the maximum of the posterior

$$\begin{aligned} & p\Big(\{B^{(i)}\}_{i=1}^{N}, \Gamma, \mathcal{P}, \Psi, \boldsymbol{\kappa} \mid \mathcal{D}\Big) \\ & \quad \propto p(\Gamma)\, \mathcal{L}_{\mathrm{con}} \mathcal{L}_{\mathrm{prog}}, \end{aligned} \tag{3}$$

where $\mathcal{L}_{\mathrm{con}}$ is the likelihood of the observed task features under the stage-wise constraint model and $\mathcal{L}_{\mathrm{prog}}$ evaluates directional progress within each candidate stage. Exact joint inference is intractable because the schedules, discrete modes, shared constraint parameters, and local distribution parameters are coupled. SWCL therefore uses an alternating approximate MAP procedure described below.

A. Constraint-specific Observation Models

For a fixed stage interval, each feature is evaluated under a mode $m \in \mathcal{M} = \{\varnothing, \mathrm{eq}, \mathrm{lb}, \mathrm{ub}\}$. Every mode defines a proper probability density for its observed feature values. The inactive mode describes ordinary local variation, whereas the three active modes express a task-level target or boundary shared across demonstrations.

1) Inactive Mode: When feature j is inactive in stage k, its values need not concentrate around a task-level shared parameter. We use a local Gaussian as the reference model. Among continuous distributions with a specified mean and variance, the Gaussian has maximum entropy; it therefore provides a minimally structured model of ordinary local feature variation

without imposing equality or one-sided boundary semantics. Its density is

$$p_{\varnothing}\left(\phi \mid \mu_{k,j}^{(i)}, \sigma_{k,j}^{(i)}\right) = \mathcal{N}\left(\phi; \mu_{k,j}^{(i)}, (\sigma_{k,j}^{(i)})^2\right), \tag{4}$$

where $\mu_{k,j}^{(i)}$ and $\sigma_{k,j}^{(i)}$ are fitted separately to each demonstration-stage interval. We estimate them robustly using the interval median and the central 80% quantile range, respectively. Consequently, inactive feature distributions may differ across demonstrations without inducing a shared task constraint.

2) Equality Mode: An equality-constrained feature is expected to remain close to a common task-level target $\eta_{k,j}$. Its observation model is

$$p_{\mathrm{eq}}\left(\phi \mid \eta_{k,j}, \sigma_{k,j}^{(i)}\right) = \mathcal{N}\left(\phi; \eta_{k,j}, (\sigma_{k,j}^{(i)})^2\right), \qquad \sigma_{k,j}^{(i)} \le \bar{\sigma}_{\mathrm{eq}}, \tag{5}$$

Unlike the inactive mode, whose location is fitted independently for each demonstration, the equality mode is centered at a task-level target $\eta_{k,j}$ shared across demonstrations. Its scale remains local to each demonstration-stage interval and is restricted by the hyperparameter $\bar{\sigma}_{\mathrm{eq}}$, which specifies how tightly observations must concentrate around the shared target for an equality interpretation to be plausible. The estimation of the shared target and local scales is described in Section IV-D.

3) Inequality Modes: We adopt the modeling assumption that, when a one-sided inequality is active and sufficiently expressed in the demonstrations, its feature values exhibit a boundary-anchored distribution: the observations remain primarily on the feasible side, while a non-negligible amount of probability mass occurs near the boundary. This pattern provides evidence for identifying both the activation and direction of the inequality. In contrast, if all demonstrations remain far inside the feasible region and never approach the true boundary, the inequality is generally not identifiable from feasible demonstrations alone [4]. In such cases, prior knowledge about its activation and direction can instead be encoded through the configurable mode probabilities $\pi_j(m)$ introduced in Section IV-B.

For an active inequality with boundary η, define the signed slack

$$y_m(\phi; \eta) = \begin{cases} \phi - \eta, & m = \mathrm{lb}, \\ \eta - \phi, & m = \mathrm{ub}. \end{cases} \tag{6}$$

Positive slack lies on the feasible side of the boundary. The inequality only restricts one side of the feature space and does not prescribe a preferred value within the feasible region. We therefore model the nonnegative slack with a half-Student-t density rather than a symmetric distribution around the boundary. Its mode at zero represents boundary contact, while its heavy tail allows demonstrations to maintain different feasible margins. For brevity, write $y = y_m(\phi; \eta)$. To accommodate observation noise and occasional small violations, we attach a soft exponential tail to the infeasible side:

$$p_m(\phi \mid \eta, \sigma) = \frac{1}{Z_{\nu,\alpha}} \begin{cases} \frac{2}{\sigma} t_\nu\left(\frac{y}{\sigma}\right), & y \ge 0, \\ \frac{2}{\sigma} t_\nu(0) \exp\left(\frac{y}{\alpha\sigma}\right), & y < 0, \end{cases} \tag{7}$$

where $m \in \{\mathrm{lb}, \mathrm{ub}\}$, t_ν is the standard Student-t density, σ is a scale estimated separately for each demonstration-stage interval, $\alpha = 0.1$ controls the boundary softness, and $Z_{\nu,\alpha} = 1 + 2\alpha t_\nu(0)$ is the normalizing constant. The heavy-tailed feasible side accommodates demonstrations that maintain different margins from the same task boundary, while the soft tail allows rare violations without assigning zero likelihood.

For a fixed boundary, σ describes how widely the feasible observations spread away from it. We choose σ so that the model and the observed slacks have the same 90th percentile, using the same rule for all tasks. Once the stage intervals are available, the shared boundary $\eta_{k,j}$ is estimated jointly across demonstrations by minimizing their aggregated inequality negative log-likelihood. Thus, $\eta_{k,j}$ represents the learned task constraint, whereas σ describes the variation within an individual demonstration-stage interval.

4) Unified Constraint Likelihood: Conditioned on the schedules and model parameters, SWCL factorizes the constraint likelihood across demonstrations, stages, features, and observations:

$$\mathcal{L}_{\mathrm{con}} = \prod_{i=1}^{N} \prod_{k=1}^{K} \prod_{j=1}^{F} \prod_{t \in \mathcal{I}_k^{(i)}} p_{\gamma_{k,j}}\left(\phi_j(x_t^{(i)}) \mid \eta_{k,j}, \psi_{k,j}^{(i)}\right), \tag{8}$$

Here, $\gamma_{k,j} \in \mathcal{M}$ denotes the single mode assigned to feature j in stage k, and $p_{\gamma_{k,j}}$ refers to the corresponding density $p_{\varnothing}$, p_{eq}, p_{lb}, or p_{ub} defined above. For each stage–feature pair, SWCL fits all four candidate densities to the observations assigned to that stage and evaluates their aggregated likelihoods across demonstrations. Figure 1 illustrates this likelihood-based comparison: each curve represents the fitted density under one candidate mode, and a lower negative log-likelihood indicates that the mode explains the observations better. In the complete objective, this likelihood evidence is combined with the configurable mode prior $\pi_j(m)$. The progress likelihood additionally influences the inferred stage intervals and therefore indirectly affects which observations enter the mode comparison. The candidate that yields the best combined objective is assigned to $\gamma_{k,j}$. Because the lower- and upper-bound modes are treated as separate candidates, this comparison determines both whether an inequality is active and its direction. The local parameters $\psi_{k,j}^{(i)}$ are those required by the selected density, while $\eta_{k,j}$ is omitted for the inactive mode. The joint update of $\gamma_{k,j}$ and $\eta_{k,j}$ is described in Section IV-D.

B. Constraint-mode Prior

Likelihood alone can favor an active mode because it provides a more specialized explanation of finite data. SWCL therefore places a prior on the mode of each feature. Let $a_j \in (0, 1)$ be the prior probability that feature j is active, and let $q_{m,j}$ be the conditional prior over active modes, with $\sum_{m \in \{\mathrm{eq,lb,ub}\}} q_{m,j} = 1$. We define

$$\pi_j(m) = \begin{cases} 1 - a_j, & m = \varnothing, \\ a_j q_{m,j}, & m \in \{\mathrm{eq}, \mathrm{lb}, \mathrm{ub}\}. \end{cases} \tag{9}$$

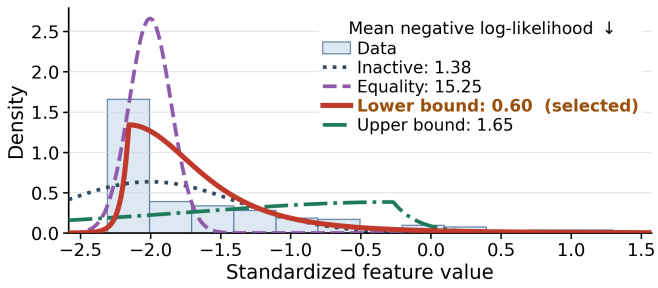


Fig. 1. Mode-specific fits to a feature distribution aggregated across demonstrations. The four curves use their respective fitted parameters, and the legend reports their mean negative log-likelihoods (lower is better). In this example, the lower-bound mode provides the strongest likelihood evidence and is selected.

The full mode prior factorizes as $p(\Gamma) = \prod_{k=1}^{K}\prod_{j=1}^{F}\pi_j(\gamma_{k,j})$. It acts at the task-model level: the prior is applied once when selecting the shared mode of a stage-feature pair rather than once per observation.

C. Directional Subgoal Progress

Constraint likelihoods characterize which feature distributions are expressed within a stage, but they do not by themselves require the motion to form a coherent segment. SWCL therefore includes an auxiliary directional-progress likelihood. A candidate interval $\mathcal{I}(s,e) = \{s,\dots,e\}$ induces the endpoint $g(\mathcal{I}) = x_e$. For each nonzero transition inside the interval, we define

$$r_t(\mathcal{I}) = \frac{\|x_t - g(\mathcal{I})\|_2 - \|x_{t+1} - g(\mathcal{I})\|_2}{\|x_{t+1} - x_t\|_2}, \qquad r_t \in [-1,1]. \tag{10}$$

A value near one indicates motion directly toward the endpoint, zero indicates approximately tangential motion, and a negative value indicates motion away from it. For stage k, these observations follow

$$p(r \mid \kappa_k) = \frac{\kappa_k \exp(\kappa_k r)}{2\sinh(\kappa_k)}, \qquad r \in [-1,1], \tag{11}$$

with the limiting uniform density $p(r \mid 0) = 1/2$. Up to the constant cost of the uniform reference density, the negative log-likelihood of an interval containing n valid transitions is

$$C_{\text{prog},k}(\mathcal{I}) = n\log\left(\frac{\sinh\kappa_k}{\kappa_k}\right) - \kappa_k\sum_{t\in\mathcal{I}} r_t(\mathcal{I}). \tag{12}$$

Thus, $\kappa_k = 0$ disables the progress contribution, whereas a larger κ_k favors intervals that move consistently toward their endpoint. This term provides temporal structure for segmentation and is not interpreted as a learned task constraint.

D. Shared Constraint Estimation

Suppose the current schedules provide interval $\mathcal{I}_k^{(i)}$ with length $n_k^{(i)}$ for every demonstration and stage. For a candidate mode m, shared active parameter η, and local distribution parameter $\psi_{k,j}^{(i)}$, define the per-demonstration mean negative log-likelihood

$$\bar{\ell}_{k,j}^{(i)}(m,\eta) = \min_{\psi_{k,j}^{(i)}} \frac{1}{n_k^{(i)}} \sum_{t\in\mathcal{I}_k^{(i)}} -\log p_m\left(\phi_j(x_t^{(i)}) \mid \eta, \psi_{k,j}^{(i)}\right). \tag{13}$$

The shared mode and parameter are updated by

$$(\hat{\gamma}_{k,j}, \hat{\eta}_{k,j}) = \arg\min_{m,\eta}\left[\sum_{i=1}^{N}\bar{\ell}_{k,j}^{(i)}(m,\eta) - \log\pi_j(m)\right], \tag{14}$$

where the optimization over η is omitted for $m = \varnothing$. Equality, lower-bound, and upper-bound candidates each use one shared target or boundary across demonstrations, while their scales are estimated separately for each demonstration. The inactive candidate similarly fits its own local location and scale for each demonstration.

Equation (14) aggregates soft likelihood evidence rather than first converting every demonstration into a hard vote. Normalizing within each demonstration gives demonstrations equal influence on the shared task model despite different sampling densities or recovered stage lengths. This normalized composite likelihood, together with locally estimated distribution parameters and hard stage schedules, is the practical surrogate used for the shared-model part of the approximate MAP inference.

The progress parameter is updated from the currently recovered intervals by maximum likelihood:

$$\hat{\kappa}_k = \arg\min_{\kappa\geq 0}\left[n_k\log\left(\frac{\sinh\kappa}{\kappa}\right) - \kappa\sum_{i,t} r_t(\mathcal{I}_k^{(i)})\right], \tag{15}$$

where n_k is the total number of valid progress observations in stage k. If their empirical mean is nonpositive, the optimum is set to $\hat{\kappa}_k = 0$. A representative shared subgoal is obtained as the geometric median of the recovered demonstration-specific stage endpoints.

E. Segmentation under the Shared Constraint Model

Conditioned on the current shared model, demonstrations decouple and can be segmented independently. For a candidate interval $\mathcal{I} = [s,e]$ assigned to stage k in demonstration i, the constraint cost is

$$C_{\text{con},k}^{(i)}(\mathcal{I}) = \sum_{j=1}^{F}\min_{\psi_{k,j}^{(i)}}\sum_{t=s}^{e} -\log p_{\hat{\gamma}_{k,j}}\left(\phi_j(x_t^{(i)}) \mid \hat{\eta}_{k,j}, \psi_{k,j}^{(i)}\right), \tag{16}$$

where $\hat{\eta}_{k,j}$ is omitted when the shared mode is inactive. Because the shared modes are fixed during this step, their prior cost is constant with respect to the candidate cutpoints and need not be included. The complete candidate-stage cost is

$$C_k^{(i)}(\mathcal{I}) = C_{\text{con},k}^{(i)}(\mathcal{I}) + C_{\text{prog},k}^{(i)}(\mathcal{I}). \tag{17}$$

Let $F_k^{(i)}(t)$ denote the minimum cost of segmenting the prefix $\{x_1^{(i)},\dots,x_t^{(i)}\}$ into the first k stages. The optimal conditional segmentation is obtained using the recurrence

$$F_k^{(i)}(t) = \min_{1\leq s\leq t}\left[F_{k-1}^{(i)}(s-1) + C_k^{(i)}([s,t])\right]. \tag{18}$$

Algorithm 1 Stage-Wise Constraint Learning

Input: demonstrations $\mathcal{D}$, number of stages K, feature bank Φ

Output: schedules $\{B^{(i)}\}$, shared modes $\hat{\Gamma}$, and active parameters $\hat{\mathcal{P}}$

```
Extract and standardize the feature trajectories
for all demonstrations i = 1, ..., N do
    Initialize B^(i) by dynamic programming with locally free constraint modes and parameters
end for
Estimate (Γ̂, P̂) using (14)
Estimate κ̂ using (15)
repeat
    for all demonstrations i = 1, ..., N do
        Compute candidate-stage costs using (17)
        Update B^(i) by (18) and backtracking
    end for
    for all stages k = 1, ..., K and features j = 1, ..., F do
        Update (γ̂_{k,j}, η̂_{k,j}) using (14)
    end for
    Update κ̂ and the representative stage subgoals
until the schedules and shared constraint model converge
```

The initialization is $F_0^{(i)}(0) = 0$ and $F_0^{(i)}(t) = +\infty$ for $t > 0$. Backtracking from $F_K^{(i)}(T_i)$ recovers the demonstration-specific stage schedule. For a fixed shared constraint model and progress parameters, this dynamic program exactly solves the ordered segmentation subproblem.

F. Alternating Inference

SWCL initializes each demonstration using a free segmentation pass. In this pass, every candidate interval locally selects its best constraint mode and parameters without enforcing a shared cross-demonstration model. The resulting schedules provide the intervals used to estimate the first shared constraint modes, parameters, and progress concentrations.

Subsequent iterations alternate between two conditional updates. First, with the shared model fixed, each demonstration is resegmented by the dynamic program in (18). Second, with the recovered intervals fixed, every stage-feature pair is updated independently using (14), and the stage-wise progress parameters are updated using (15). The procedure terminates when both the stage schedules and shared prototypes stabilize, or when the change in the practical objective falls below a prescribed tolerance. Because the algorithm uses hard schedules, locally estimated distribution parameters, and a demonstration-normalized shared update, it should be interpreted as approximate MAP inference rather than exact posterior integration or exact coordinate ascent on one raw-sample likelihood.

V. Simulation Experiment

A. Experimental Setup

1) Benchmark Datasets: We evaluate SWCL on three simulated multi-stage benchmark tasks: `S3ObsAvoid`, `S4SlideInsert`, and `S5SphereInspect` with three baselines. The tasks contain three, four, and five ordered stages, respectively, and are designed to evaluate stage-wise constraint-mode identification, and constraint-parameter estimation.

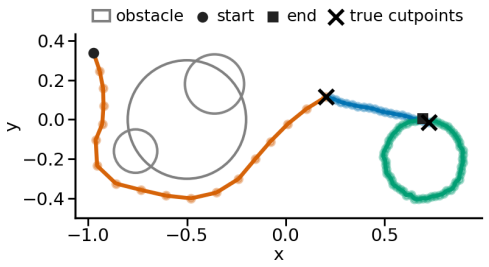


Fig. 2. Example demonstration from the `S3ObsAvoid` environment. The colored trajectory segments correspond to the three ground-truth stages: obstacle avoidance, transition, and terminal arc following. Gray ellipses denote the composite obstacle, the black circle and square mark the start and end states, and black crosses indicate the true stage cutpoints.

S3ObsAvoid. A three-stage planar navigation task: obstacle avoidance → transition → terminal arc following. Fig. 2 shows a representative demonstration and its ground-truth stage cutpoints.

S4SlideInsert. A four-stage slide-and-insert task: free-space approach → surface contact and alignment → guided sliding → final insertion. The task is performed by a UR5 arm simulated in PyBullet.

S5SphereInspect. A five-stage spherical inspection task: approach → surface inspection → transition → outer-shell inspection → departure. The task is also performed by a simulated UR5 arm, and Fig. 7 visualizes its five stages.

The feature banks of the three tasks are summarized in Table I, while their ground-truth and learned stage-wise constraints are compared in Fig. 5. Only a subset of the exposed features is constrained in each stage; the remaining features are included to evaluate whether the method can correctly identify inactive or decoy features. All methods are trained on the same 10 demonstrations per task, generated as feasible planned trajectories satisfying the task structure in (2). All code and datasets are available at https://github.com/epfl-lasa/Constraints_Learning_Learn_Stage-wise_Constraints_from_Demonstrations.

2) Baselines: As discussed in the related work section, most existing constraint-learning methods are not designed to recover multi-stage constraints directly. Therefore, we adapt three representative segmentation paradigms to the stage-wise constraint learning setting.

It is important to note that these baselines do not explicitly recover the full constraint mode matrix Γ. In particular, they are not designed to determine which stage-feature pairs are active or inactive. Therefore, when computing constraint-parameter errors for the baselines, we provide the ground-truth active stage-feature pairs and compare only the parameters fitted on those pairs. For inequality-type features, the ground-truth lower/upper direction is also provided during parameter extraction. This protocol is favorable to the baselines, since SWCL must infer both the active pairs and the inequality

Feature	Description	Unit
S3ObsAvoid		
obs_dist	Distance to the obstacle	cm
speed	Speed magnitude	cm/s
arc_dist	Distance to the terminal-circle center	cm
heading	Heading angle	rad
line_dist	Distance to an irrelevant straight line	cm
noise	Sine noise	–
S4SlideInsert		
surf_dist	Distance to the table contact surface	mm
center_dist	Absolute lateral distance from the slot centerline	mm
orient_err	Absolute orientation error relative to the slot	deg
speed	Speed magnitude of the tool	mm/s
normal_force	Normal contact force applied against the guide	N
start_dist	Distance to the demonstration start pose	mm
insert_err	Remaining insertion distance along the slot	mm
S5SphereInspect		
surf_dist	Radial distance to the sphere surface	mm
normal_err	Angle between the tool axis and the sphere normal	deg
speed	Speed magnitude of the tool	mm/s
ang_speed	Angular speed magnitude	deg/s
start_dist	Distance to the demonstration start position	mm
goal_dist	Distance to the nominal inspection goal	mm

TABLE I
FEATURE BANKS USED IN THE THREE BENCHMARK TASKS. UNITS REFER TO THE FEATURE VALUES BEFORE STANDARDIZATION.

directions automatically.

Feature-constrained Hidden Markov Model (FCHMM). This baseline is adapted from the classical Gaussian-emission HMM segmentation pipeline [11]. We adapt it to perform segmentation and constraint modeling jointly. Specifically, each latent stage is augmented with explicit feature-wise constraint emissions. For equality-type features, the emission is modeled by a Student-t distribution, whose fitted center is extracted as the learned equality parameter. For inequality-type features, we use the same soft half-t local fitting rule as SWCL, but with the ground-truth inequality direction provided. The fitted boundary is extracted as the learned inequality parameter.

Autoregressive Hidden Semi-Markov Model (ARHSMM). This baseline first segments the demonstrations using an autoregressive hidden semi-Markov model [12], where the autoregressive observation model is fitted on the joint state-feature representation $(x_t, \phi(x_t))$. After segmentation, it performs a post-hoc constraint-parameter fitting step on the recovered segments using the ground-truth active stage-feature pairs. For equality-type features, the parameter is estimated by the segment median. For inequality-type features, the boundary is estimated from the corresponding tail quantile using the ground-truth lower/upper direction. It therefore represents a *segmentation-first, constraint-fitting-second* pipeline, which contrasts with the joint learning pipeline of SWCL and FCHMM.

Ordered clustering (Cluster). This baseline segments demonstrations by ordered clustering in the joint state-feature space, following the general idea of clustering-based trajectory segmentation [29]. After segmentation, it fits stage-wise constraint parameters on the recovered segments using the same post-hoc fitting protocol as ARHSMM.

3) Evaluation metrics: We evaluate the four methods mainly on constraint recovery quality and secondly on segmentation quality.

Normalized constraint error. For each truly active stage-feature pair, we compare the learned constraint parameter against the ground-truth target or boundary value. Since different features have different physical scales, the absolute error is normalized by a feature-specific scale estimated from the demonstrations and averaged across all truly constrained stage-feature pairs. Lower values indicate more accurate constraint recovery. Truly inactive stage-feature pairs are excluded from this metric. For SWCL, we additionally inspect the learned constraint mode matrix and the corresponding recovered parameter values. While the other three baselines directly use ground truth constraint mode and only compare constraint values.

Mean absolute cutpoint error. For each demonstration, we compare each predicted stage cutpoint with the corresponding ground-truth cutpoint and compute the absolute boundary error. We first average these errors over all cutpoints within the same demonstration, and then average the resulting per-demonstration errors over all demonstrations. Lower values indicate more accurate segmentation.

B. Results

1) Main benchmark comparison: Figure 4 compares SWCL with the three baselines on the benchmark datasets. The top row reports mean absolute cutpoint error and the bottom row reports normalized constraint error. The three baselines are trained from 10 random initializations and the reported results are averaged across these runs. In contrast, SWCL is run once, since it does not depend on random initialization. The parameters of SWCL and the baselines are tuned by sweeping.

SWCL achieves the lowest error on both metrics across all three datasets. In terms of segmentation, SWCL recovers substantially more accurate cutpoints than the baselines, especially on `S3ObsAvoid` and `S5SphereInspect`. The advantage is also reflected in constraint recovery: SWCL obtains the lowest normalized constraint error in all tasks, with the largest gap appearing on `S5SphereInspect`, where the baselines struggle to recover the correct stage-wise constraint values. Fig. 3 shows the normalized feature profiles of one `S5SphereInspect` demonstration together with the SWCL-learned and ground-truth cutpoints. The learned cutpoints are close to the true stage cutpoints and align with visible changes in the feature regimes.

Among the baselines, none performs consistently well across both metrics and all datasets. FCHMM sometimes achieves reasonable segmentation accuracy, but its constraint error remains higher than SWCL, especially on `S5SphereInspect`. ARHSMM generally gives larger cutpoint errors and also has relatively high constraint error, suggesting that segmentation based mainly on autoregressive temporal structure does not align well with the constraint-relevant stages in these tasks. Cluster obtains competitive cutpoint error on some datasets, but its constraint recovery remains less accurate than SWCL because the constraint parameters are fitted only after segmentation.

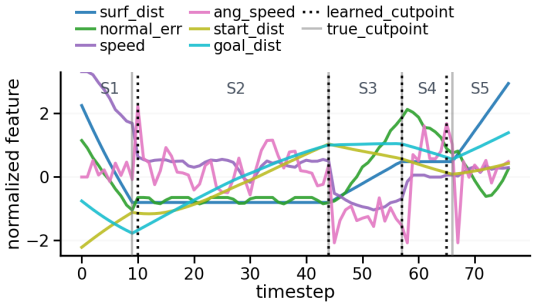


Fig. 3. Normalized feature profiles of one representative `S5SphereInspect` demonstration. Solid gray vertical lines indicate the ground-truth cutpoints, and dotted black vertical lines indicate the cutpoints learned by SWCL.

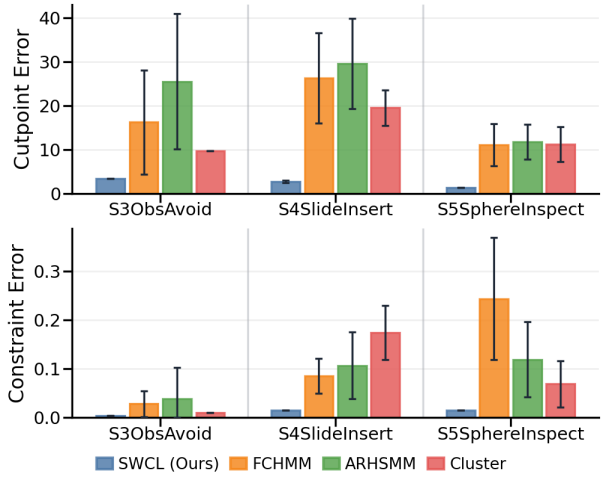


Fig. 4. Main benchmark comparison across the three datasets. Top: mean absolute cutpoint error. Bottom: normalized constraint error. Lower is better in both plots.

A second visible pattern is variance. ARHSMM and FCHMM exhibit large error bars on several datasets, indicating substantial sensitivity to random initialization or unstable segmentation. By contrast, SWCL has much smaller variability because each segmentation update is solved by a full dynamic-programming search conditioned on the current shared prototypes. These results support a major benefit of SWCL: coupling local constraint evidence, subgoal progress, and cross-demonstration consensus leads to more reliable recovery of both stage cutpoints and stage-wise constraints.

2) Learned constraint modes of SWCL: Beyond the scalar error metrics, we directly examine the stage-wise constraint structure recovered by SWCL. Fig. 5 compares the ground-truth constraint matrix with the learned shared matrix $\hat{\Gamma} = [\hat{\gamma}_{k,j}]$ for each benchmark task. Each cell represents one stage-feature pair and is labeled as inactive, equality-constrained, lower-bound constrained, or upper-bound constrained. For active entries, the cell text additionally reports the corresponding equality target or inequality boundary in the physical unit shown on the vertical axis.

SWCL recovers the correct full constraint-mode structure on `S3ObsAvoid` and `S5SphereInspect`. On `S4SlideInsert`, most modes are also recovered correctly. The main mismatch is that SWCL additionally identifies a speed upper-bound constraint in stage 2, where the ground-truth mode is inactive. This likely occurs because the motion slows down from stage 1 to stage 2, making the local speed profile appear one-sided to be interpreted as an active inequality constraint. Overall, the learned constraint matrices show that SWCL recovers the dominant task-level constraint structure, while also producing interpretable numerical constraint values for the active stage-feature pairs.

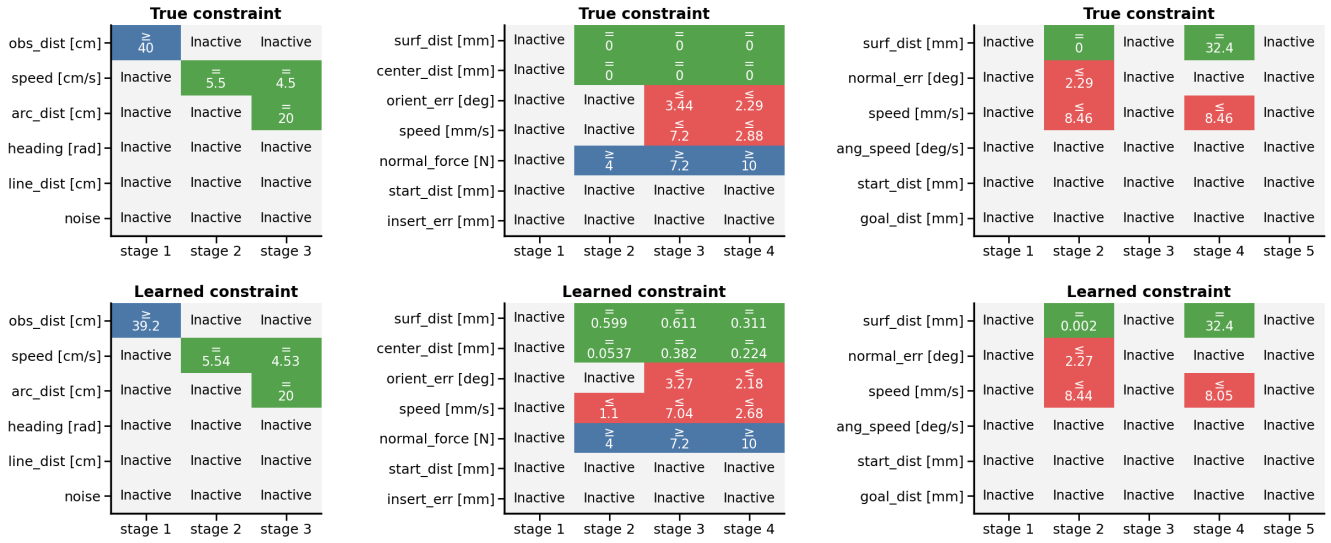
C. Reuse the Learned Constraints for Planning

We further evaluate whether the learned stage-wise constraints can be reused for planning. Given the learned constraint mode prototype $\hat{\Gamma}$ and parameter prototypes $\{\hat{\eta}_{k,j}\}$, we construct new task instances with prescribed start states, final goals, and trajectory lengths, and use the learned active constraints as stage-wise feasibility conditions in (2). Different from the formulation in (2), we do not prescribe intermediate subgoals during planning. Instead, we specify the length of each stage according to the average stage-length ratio observed in the demonstrations, which determines the stage schedule B. We also add smoothness as the objective function to (2), while enforcing the learned stage-wise constraints and the known feasibility requirements in $\mathcal{C}_{\text{known}}$. In this task, $\mathcal{C}_{\text{known}}$ mainly includes global task-agnostic velocity limits, which impose absolute feasibility bounds on the generated trajectory. These are different from the learned stage-wise speed constraints, which encode task-specific motion requirements in particular stages. The planned task-space trajectory is then converted to UR5 joint targets by inverse kinematics and tracked in PyBullet using its joint `TORQUE_CONTROL` mode in `S4SlideInsert`, and `POSITION_CONTROL` mode in `S5SphereInspect`.

For `S4SlideInsert`, we test transferring the learned constraints to two new environments that preserve the same stage-wise constraint semantics but modify the environment. While the original task uses a straight guide rail and a horizontal surface, the first transfer task uses a tilted table surface, and the second modifies the straight guide rail to a curved guide rail. In these transfer tasks, the corresponding feature functions are given and are defined with respect to the modified geometry; SWCL transfers only the learned stage-wise constraint modes and parameters. Figure 6 shows the two transfer tasks with the learned constraints.

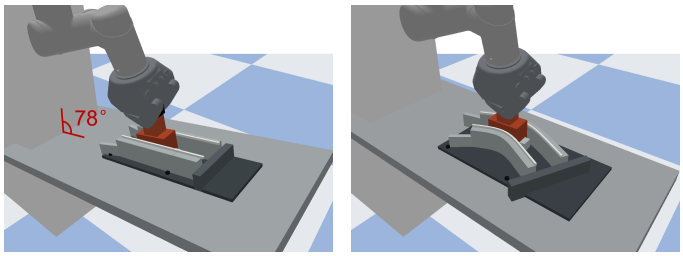
For `S5SphereInspect`, we plan and execute new trajectories in the same spherical inspection environment but with different start and terminal configurations. Figure 7 shows five snapshots from one PyBullet execution. The planned trajectory follows the learned stage-wise constraints while adapting the surface-inspection and near-surface inspection stages to the new task instance.

To quantify execution feasibility, we compute the per-step violation rate of the true stage-wise constraints. Specifically, for each timestep in each stage, we check whether any truly active constraints are violated and report the fraction of timesteps that violate any true constraint. The violation rates are 4.37% for the tilted-table `S4SlideInsert` transfer, 0.61% for



(a) `S3ObsAvoid` (b) `S4SlideInsert` (c) `S5SphereInspect`

Fig. 5. Comparison of the ground-truth and learned stage-wise constraint matrices for the three benchmark tasks. In each panel, the upper matrix shows the ground-truth constraints and the lower matrix shows the constraints recovered by SWCL. Each cell corresponds to one stage-feature pair. Colors indicate the constraint mode, while the cell text reports the equality target or inequality boundary in the physical units shown on the vertical axis. Inactive entries indicate unconstrained stage-feature pairs.



(a) Tilted surface transfer (b) Curved rail transfer

Fig. 6. Transfer of learned `S4SlideInsert` constraints to two modified tasks with the same stage-wise constraint semantics but different environments. (a) The guide surface is tilted. (b) The straight guide is replaced by a curved rail.

the curved-rail `S4SlideInsert` transfer, and 0.11% for `S5SphereInspect`. These results indicate that the learned constraints can be used as reusable planning constraints even beyond the original demonstrations. The video of the all three experiments is attached as supplementary material.

VI. Real-robot Validation

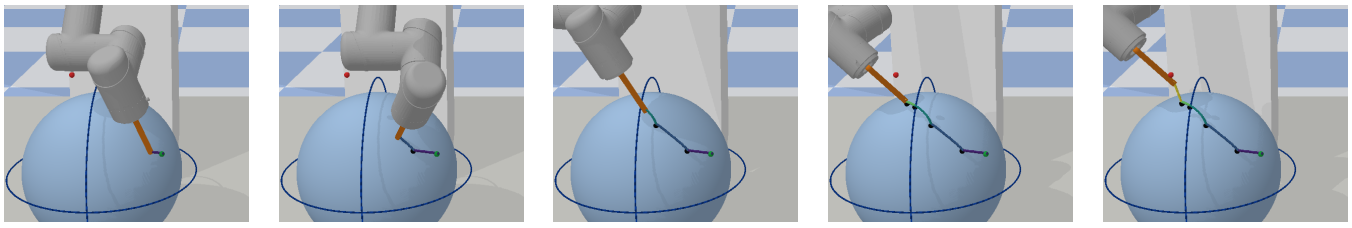
A. Experiment Setup and Constraint Learning

We then apply our method to an automated bar-clearing task in production lines. A bar-shaped aluminum workpiece is placed on a conveyor, with several small objects remaining on its upper surface. Before the workpiece proceeds to downstream processing, the robot must clear these objects. The task consists of five stages: approach the bar end while avoiding the obstacle → sweep objects off the bar surface → reposition → sweep objects off the table → departure. Fig. 9 illustrates the task and the first four stages. The task is performed by a KUKA LBR iiwa 14 equipped with a custom sweeping tool. The conveyor is static during execution. The experiment video is attached to the submission.

Seven features are considered. The translational features comprise the obstacle clearance (`obs_dist`), the end-effector height above the table (`table_dist`), the lateral offset from the bar centerline (`lateral_offset`), and the displacement along the bar axis from one bar end (`axial_offset`). The tool orientation is described by its pitch (`tool_pitch`), roll (`tool_roll`), and yaw (`tool_yaw`) relative to the same bar-fixed frame.

A human demonstrator provided six kinesthetic demonstrations, from which the stage-wise constraint model is learned and summarized in Fig. 8. The method recovered equality constraints for both sweeping stages (stages 2 and 4). In both stages, the tool maintains a heading approximately perpendicular to the bar; stage 2 additionally maintains a nearly constant lateral offset, whereas stage 4 maintains a nearly constant axial offset. The method also recovered the lower-bound obstacle-clearance constraint and its threshold in stage 1. No feature constraints are activated in stages 3 and 5.

The learned model contains a potentially redundant lower bound on `obs_dist` in stage 4. During data collection, the obstacle stayed near one end of the bar. As the stage-4 motion started from the opposite end and proceeded approximately perpendicular to the bar, its distance from the obstacle naturally exhibited a one-sided distribution that resembled a lower-bound constraint. This term is therefore consistent with the demonstrations, although it may be unnecessary. Given only the feasible demonstrations and the information assumed in this work, a genuine inequality constraint and an incidental one-sided distribution are observationally indistinguishable. This is an inherent identifiability limitation under incomplete information. However, we verified that, when stronger prior



(a) stage 1: approach (b) stage 2: surface inspection (c) stage 3: transition (d) stage 4: outer-shell inspection (e) stage 5: departure

Fig. 7. PyBullet execution snapshots of a new `S5SphereInspect` planning instance using the learned stage-wise constraints. The five snapshots correspond to the five stages of the task: (a) the tool approaches the sphere from a new start configuration; (b) it performs surface inspection while maintaining the learned surface-distance, normal-alignment, and speed constraints; (c) it transitions away from the strongly constrained surface-tracing regime; (d) it performs inspection on a larger spherical shell farther from the sphere; and (e) it departs toward the terminal goal.

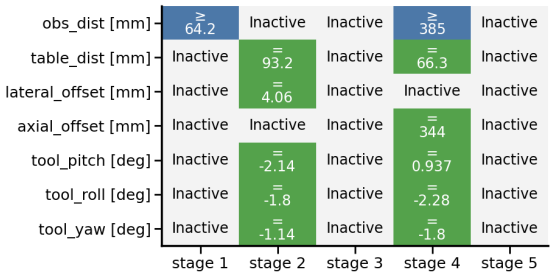


Fig. 8. Stage-wise constraints learned for the real-robot bar-clearing task.

knowledge is provided, assigning a lower constraint activation prior to the stage-4 `obs_dist` can remove this term while keeping other constraints unchanged.

B. Planning and Execution

We reuse the learned stage-wise constraint model in an optimization-based planner. For this experiment, the state $x_t \in \mathcal{X}$ is the end-effector pose expressed in a bar frame, i.e., $x_t = (p_t, R_t)$, where $p_t \in \mathbb{R}^3$ and $R_t \in SO(3)$ denote position and orientation relative to the bar. The start state, goal state and subgoals are represented in the same frame.

Let $(\widehat{\Gamma}, \widehat{\mathcal{P}})$ be the constraint model recovered from the demonstrations, with learned modes $\widehat{\gamma}_{k,j}$ and parameters $\widehat{\eta}_{k,j}$. For each stage, the subgoal g_k is set to the average of the corresponding stage endpoints over all demonstrations. Whereas the final subgoal is the specified terminal state, $g_K = x_{\text{goal}}$. To use the learned equality and inequality constraints in a common objective, we define the violation residual

$$r_{k,j}(x) = \begin{cases} 0, & \widehat{\gamma}_{k,j} = \varnothing, \\ \phi_j(x) - \widehat{\eta}_{k,j}, & \widehat{\gamma}_{k,j} = \text{eq}, \\ [\widehat{\eta}_{k,j} - \phi_j(x)]_+, & \widehat{\gamma}_{k,j} = \text{lb}, \\ [\phi_j(x) - \widehat{\eta}_{k,j}]_+, & \widehat{\gamma}_{k,j} = \text{ub}, \end{cases} \tag{19}$$

where $[z]_+ = \max(z, 0)$. Thus, an equality constraint is penalized according to its deviation from the learned target, while an inequality constraint incurs a penalty only on its infeasible side.

The planner discretizes each stage according to a fixed spatial waypoint spacing. If stage k contains T_k waypoints, its boundary is $b_k = 1 + \sum_{l=1}^{k} T_l$. Note that, B specifies only the waypoint allocation across stages and does not prescribe their execution duration. Planning is then formulated as

$$\begin{aligned} \min_{\tau=\{x_t\}_{t=1}^{T}} \quad & J_{\text{smooth}}(\tau) + \lambda_c \sum_{k=1}^{K} \sum_{t \in \mathcal{I}_k(B)} \sum_{j=1}^{F} r_{k,j}(x_t)^2 \\ \text{s.t.} \quad & x_1 = x_{\text{start}}, \\ & x_{b_k - 1} = g_k, \qquad k = 1, \ldots, K, \\ & \tau \in \mathcal{C}_{\text{known}}. \end{aligned} \tag{20}$$

Here, J_{smooth} penalizes path length and rapid changes in end-effector position and orientation, and λ_c is a large weight penalizing constraint violation. $\mathcal{C}_{\text{known}}$ specifies the known workspace limit. The problem is therefore a nonlinear optimization problem and is solved with a SciPy library.

After optimization, the task-space waypoints are converted to joint-space waypoints using sequential inverse kinematics (IK). The IK solution for each waypoint is initialized from the previous waypoint so that the resulting joint path remains on a consistent solution branch. The joint path is then time-parameterized using the average speed in each stage of the demonstrations. The generated trajectory contains joint positions, velocities, accelerations, and timestamps, and is sent to a standard ROS joint-position trajectory controller. The controller interpolates the time-parameterized joint trajectory and communicates the commands to the robot through the position-control interface at 200 Hz.

C. Results

We evaluate whether the learned constraints support successful task execution on the real robot using three task-level criteria: obstacle avoidance, sweeping the objects off the bar, and sweeping the objects off the table. We varied the positions and orientations of both the obstacle and the bar and evaluated 20 execution trials. The robot passed the obstacle in 18 of 20 trials (90%); the two failures were caused by excessively tilted initial tool poses. It successfully swept the objects off the bar in all 20 trials (100%) and swept the objects off the table in 17 of 20 trials (85%). The latter failures occurred when too many objects were placed or when the objects can easily roll away on the table.

The learned constraints can also be transferred to related environments. As shown in Fig. 10, we replace the straight



(a) stage 1: approach (b) stage 2: bar sweeping (c) stage 3: repositioning (d) Stage 4: table sweeping

Fig. 9. Representative snapshots from a real-robot execution of the bar-clearing task using the learned stage-wise constraints. The four snapshots show the first four stages: (a) approaching the bar while avoiding the black obstacle; (b) sweeping the litter objects from the bar onto the table; (c) repositioning the tool between the two sweeping motions; and (d) sweeping the litter objects off the table. The final departure stage is omitted.

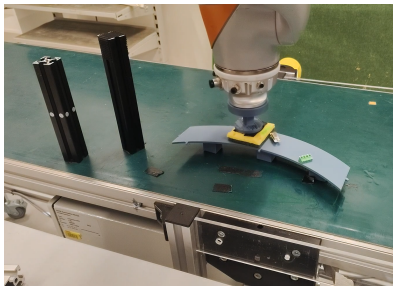


Fig. 10. Transfer setting with a curved bar and an additional obstacle.

bar with a curved bar and introduce an additional obstacle. The geometry of the curved bar is known and is used to compute the corresponding bar-relative features, while the learned stage-wise constraint model remains unchanged. `obs_dist` is evaluated with respect to the nearest of the two obstacles. The robot successfully completes the bar-clearing task in this transferred setting and examples are provided in the video.

VII. Conclusion

This paper studied stage-wise constraint learning from feasible demonstrations, where different feature constraints may become active in different stages and the stage cutpoints are unobserved. We represented the shared structure by a constraint mode matrix, whose entries indicate whether each stage-feature pair is inactive, equality-constrained, lower-bound constrained, or upper-bound constrained, together with scalar parameters for active constraints. We proposed SWCL, a method that evaluates candidate segments using local constraint evidence, subgoal-progress consistency, and cross-demonstration consensus. Conditioned on the current prototypes, each demonstration is segmented by dynamic programming, and the shared constraint modes and parameters are updated from the recovered segments.

Experiments on three simulated benchmark tasks showed that SWCL recovers more accurate stage cutpoints and constraint parameters than HMM-, HSMM-, and clustering-based baselines. The learned constraint mode matrices further show that SWCL can identify stage-dependent constraint structures across demonstrations. Planning experiments also demonstrated that the learned constraints can be reused as feasibility conditions for modified or new task instances.

The method still assumes a known number of stages. The known-stage assumption could be relaxed by allowing the dynamic-programming procedure to choose the number of stages automatically, which is an interesting direction to explore in the future. A possible extension is to add a fixed penalty for introducing each new stage: the optimizer would then create an additional segment only when the reduction in the stage-wise segmentation cost is large enough to compensate for this penalty.

References

[1] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and Autonomous Systems*, vol. 57, no. 5, pp. 469–483, 2009.

[2] D. Berenson, S. S. Srinivasa, and J. Kuffner, "Task space regions: A framework for pose-constrained manipulation planning," *The International Journal of Robotics Research*, vol. 30, no. 12, pp. 1435–1460, 2011.

[3] Z. Kingston, M. Moll, and L. E. Kavraki, "Sampling-based methods for motion planning with constraints," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 159–185, 2018.

[4] G. Chou, N. Ozay, and D. Berenson, "Learning constraints from locally-optimal demonstrations under cost function uncertainty," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3682–3690, 2020.

[5] ——, "Learning parametric constraints in high dimensions from demonstrations," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 1211–1230.

[6] G. Chou, H. Wang, and D. Berenson, "Gaussian process constraint learning for scalable chance-constrained motion planning from demonstrations," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3827–3834, 2022.

[7] D. R. R. Scobee and S. S. Sastry, "Maximum likelihood constraint inference for inverse reinforcement learning," in *International Conference on Learning Representations*, 2020.

[8] S. Malik, U. Anwar, A. Aghasi, and A. Ahmed, "Inverse constrained reinforcement learning," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 7390–7399.

[9] G. Liu, Y. Luo, A. Gaurav, K. Rezaee, and P. Poupart, "Benchmarking constraint inference in inverse reinforcement learning," in *International Conference on Learning Representations*, 2023.

[10] B. Peng and A. Billard, "Positive-unlabeled constraint learning for inferring nonlinear continuous constraints functions from expert demonstrations," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1593–1600, 2025.

[11] L. R. Rabiner, "A tutorial on hidden markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[12] M. J. Johnson and A. S. Willsky, "Bayesian nonparametric hidden semi-markov models," *Journal of Machine Learning Research*, vol. 14, no. 21, pp. 673–701, 2013.

[13] C. Truong, L. Oudre, and N. Vayatis, "Selective review of offline change point detection methods," *Signal Processing*, vol. 167, p. 107299, 2020.

[14] S. Aminikhanghahi and D. J. Cook, "A survey of methods for time series change point detection," *Knowledge and Information Systems*, vol. 51, no. 2, pp. 339–367, 2017.

[15] S. Niekum, S. Osentoski, G. D. Konidaris, S. Chitta, B. Marthi, and A. G. Barto, "Learning grounded finite-state representations from unstructured demonstrations," *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 131–157, 2015.
[16] J. R. Medina and A. Billard, "Learning stable task sequences from demonstration with linear parameter varying systems and hidden markov models," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 78. PMLR, 2017, pp. 175–184.
[17] A. K. Tanwani and S. Calinon, "Learning robot manipulation tasks with task-parameterized semi-tied hidden semi-markov model," *IEEE Robotics and Automation Letters*, vol. 1, no. 1, pp. 235–242, 2016.
[18] K. Shiarlis, M. Wulfmeier, S. Salter, S. Whiteson, and I. Posner, "TACO: Learning task decomposition via temporal alignment for control," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 4654–4663.
[19] T. Kipf, Y. Li, H. Dai, V. Zambaldi, A. Sanchez-Gonzalez, E. Grefenstette, P. Kohli, and P. Battaglia, "CompILE: Compositional imitation learning and execution," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 3418–3428.
[20] G. Sutanto, I. R. Fernández, P. Englert, R. K. Ramachandran, and G. Sukhatme, "Learning equality constraints for motion planning on manifolds," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 2292–2305.
[21] A. L. P. Ureche, K. Umezawa, Y. Nakamura, and A. Billard, "Task parameterization using continuous constraints extracted from human demonstrations," *IEEE Transactions on Robotics*, vol. 31, no. 6, pp. 1458–1471, 2015.
[22] D. Park, M. Noseworthy, R. Paul, S. Roy, and N. Roy, "Inferring task goals and constraints using bayesian nonparametric inverse reinforcement learning," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 1005–1014.
[23] J. Loula, K. Allen, T. Silver, and J. Tenenbaum, "Learning constraint-based planning models from demonstrations," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 5410–5416.
[24] C. Pérez-D'Arpino and J. A. Shah, "C-LEARN: Learning geometric constraints from demonstrations for multi-step manipulation in shared autonomy," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2017, pp. 4058–4065.
[25] A. Conkey and T. Hermans, "Learning task constraints from demonstration for hybrid force/position control," in *Proceedings of the IEEE-RAS 19th International Conference on Humanoid Robots*, 2019, pp. 162–169.
[26] C. Hegeler, F. Rozzi, L. Roveda, and K. Haninger, "Teaching contact-rich tasks from visual demonstrations by constraint extraction," 2023.
[27] A. K. Tanwani, A. Yan, J. Lee, S. Calinon, and K. Goldberg, "Sequential robot imitation learning from observations," *The International Journal of Robotics Research*, vol. 40, no. 10–11, pp. 1306–1325, 2021.
[28] H. Fu, P. Sharma, E. Stengel-Eskin, G. Konidaris, N. Le Roux, M.-A. Côté, and X. Yuan, "Language-guided skill learning with temporal variational inference," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 14 135–14 156.
[29] E. Keogh, S. Chu, D. Hart, and M. Pazzani, "Segmenting time series: A survey and novel approach," in *Data Mining in Time Series Databases*, M. Last, A. Kandel, and H. Bunke, Eds. Singapore: World Scientific, 2004, pp. 1–21.